WEEK 02

Synchronization

Problem Set

Four problems. Warm up, then make some mayhem.

This week is about taking the broken programs from Week 1 and making them work — using mutexes, condition variables, and the synchronization patterns of real production code.

Watch for the new bugs that fixing things creates.

PROBLEMS

01	Fix last week's mess	WARM UP
02	Thread-safe bounded buffer	REQUIRED
03	Thread-safe linked list	RECOMMENDED
04	Dining philosophers	STRETCH

OVERVIEW

The week, in four problems.

Four problems for the week. You'll start by fixing last week's broken examples, then build progressively more sophisticated thread-safe data structures, and end with a real coordination puzzle.

#	Problem	Type	Required?
01	Fix last week's mess	Modify + observe	Yes
02	Thread-safe bounded buffer	From scratch	Yes
03	Thread-safe linked list	From scratch	Recommended
04	Dining philosophers	Classic puzzle	Stretch

Before you start: *work through* `theory.pdf` *and run the examples in* `code/` *at least once. Pay particular attention to the producer-consumer example — its structure shows up in problems 2 and 3.*

How to compile and run

The four example programs in `code/` can be built with the provided Makefile:

```
$ cd week-2/code
$ make             # builds all four examples
$ ./counter_fixed
$ ./atomic_counter
$ ./producer_consumer
$ ./deadlock_demo  # WARNING: hangs forever - Ctrl+C to kill
$ make run-fixed   # builds and runs the three that terminate
$ make clean       # remove compiled binaries
```

Or manually: `gcc -Wall -pthread -std=c11 01_counter_fixed.c -o counter_fixed`

If your program hangs *when you run it: it's almost certainly an unreleased lock or a deadlock. Add print statements around your lock/unlock calls to see which thread is stuck where. This is the new normal for the rest of the project.*

PROBLEM 01

Fix last week's mess

WARM-UP · REQUIRED

Take two of the broken programs from Week 1 and make them actually correct using mutexes.

What to do

Part A — parallel_sum_fixed.c. Start from `03_parallel_sum.c` in last week's `code/` folder. Add a `pthread_mutex_t` that protects the shared `total`. The fixed program should produce exactly `ARRAY_SIZE` every single run, no matter how many threads you use.

Part B — bank_chaos_fixed.c. Start from `04_bank_chaos.c`. Protect the check-then-act with a mutex. The balance should never go negative, and the successful + rejected counts should add up sensibly with respect to the starting balance.

Run each at least 20 times

Single correct runs prove nothing — last week's racy versions occasionally got the right answer too. Convince yourself that *every* run is now correct:

```
$ for i in {1..20}; do ./parallel_sum_fixed; done
$ for i in {1..20}; do ./bank_chaos_fixed; done
```

Note down in your report whether you see any variation in the outputs across runs.

Reflection question

> *Time the fixed parallel_sum against the original racy version (`time ./parallel_sum_fixed`). The fixed version will be slower. Why? Where is the time going? Write a sentence or two in your report.*

What to submit

`parallel_sum_fixed.c` and `bank_chaos_fixed.c`, plus the observations and reflection answer in your report.

PROBLEM 02

Thread-safe bounded buffer

REQUIRED

Implement a generic thread-safe bounded buffer, then write a small test that exercises it with multiple producers and multiple consumers.

Requirements

Your buffer should support these operations:

```
void buffer_init(int capacity);
void buffer_put(int item);     /* blocks if full */
int  buffer_get(void);          /* blocks if empty */
void buffer_destroy(void);
```

• **buffer_put** blocks (using a condition variable) if the buffer is full, until space becomes available.

• **buffer_get** blocks if the buffer is empty, until an item arrives.

• Both operations are safe to call from multiple threads simultaneously.

Use the producer-consumer example (`03_producer_consumer.c`) as your starting point. The main difference: your version is generic — any caller can put or get without knowing about the internal synchronization.

The test

In `main`, create at least **3 producer threads** and **2 consumer threads**. Each producer produces a known sequence of integers (for example, producer N produces numbers N*1000 through N*1000+999). Each consumer consumes items and accumulates them.

When all producers are done and the buffer is drained, verify:

• The total number of items consumed equals the total number produced.

• The sum of all consumed items equals the sum of all produced items.

If either check fails, you have a bug. Fix it.

Edge cases worth considering

• What if the buffer capacity is 1? Does your code still work? It should — a capacity-1 buffer just means producer and consumer are tightly synchronized.

• What if you signal vs broadcast? Try both. Does it affect correctness? Does it affect performance with multiple consumers?

What to submit

`bounded_buffer.c` with both your buffer implementation and your multi-producer/multi-consumer test in one file, plus a paragraph in your report describing how you tested it and what edge cases you

considered.

PROBLEM 03

Thread-safe linked list

RECOMMENDED

Build a singly-linked list of integers that can be safely modified by multiple threads at the same time.

API

```
void list_init(void);
void list_insert(int value);
int  list_contains(int value);    /* returns 1 if found, 0 if not */
void list_remove(int value);
void list_destroy(void);
```

All four data-modifying operations must be safe to call concurrently from any number of threads. For this problem, **a single coarse-grained lock** protecting the whole list is acceptable — and recommended. Don't try to write a fine-grained per-node-lock version unless you finish the rest of the week's problems with time to spare; fine-grained list locking is a famously subtle topic.

The test

Spawn **8 worker threads**, each of which performs a mixture of insertions, lookups, and removals on the shared list — say 1000 operations per thread. Choose values randomly from a small range (e.g., 0–99) so threads frequently interact on the same values.

After all threads finish, verify:

- The program didn't crash.
- The list is still well-formed (you can walk from head to NULL without segfaulting or looping forever).
- `list_contains` on a value you just inserted returns 1, on one you never inserted returns 0.

Common pitfall

> ***Where does the lock live?*** *A common bug is to put the lock inside each list node. That doesn't protect operations that span multiple nodes (like insert or remove), and it doesn't protect the list head pointer at all. Use one lock for the whole list, stored alongside (or pointed to by) the list head.*

What to submit

`safe_list.c` with both the list implementation and the stress test in one file, plus a brief note in your report about how you verified correctness.

PROBLEM 04

Dining philosophers

STRETCH

The classic concurrency puzzle. Five philosophers sit around a circular table. Between each pair of adjacent philosophers is one fork. A philosopher needs **both** forks adjacent to them to eat. They spend their lives alternating between thinking, picking up forks, eating, and putting forks down.

The naive solution — every philosopher picks up their left fork first, then their right — deadlocks the moment all five philosophers grab their left fork simultaneously. Now nobody can pick up a right fork; nobody will ever release their left.

Your task

Write a program that simulates 5 philosophers as 5 threads, each eating **at least 10 times** before they exit. The 5 forks are `pthread_mutex_t`s.

First, write the broken version. Each philosopher locks their left fork, then their right. Run it. Observe the deadlock — within a second or two, all output stops. Take a note of what you see.

Then, fix it. Apply **lock ordering**: number the forks 0 through 4 and have every philosopher acquire the lower-numbered fork first, regardless of which side it's on. Now there's no cycle, no deadlock. Verify by running long enough (5+ seconds) that the simulation clearly makes progress.

Bonus: starvation

Run the fixed version for a long time and print how many times each philosopher ate. Are the counts roughly equal? If one philosopher eats far less than the others, that's **starvation** — the system makes progress as a whole, but one thread is being unfairly outpaced. Note this in your report; fixing starvation properly is genuinely hard and beyond the scope of this week.

What to submit

`philosophers.c` with your fixed (deadlock-free) version, plus a description in your report of:

- What the broken version did when you ran it (output, how it died)
- What you changed to fix it
- (Optional) Whether you saw any starvation

WHEN YOU'RE DONE

Look at what you built

You just wrote thread-safe data structures and solved one of the most famous coordination problems in computer science. The tools from this week — mutexes, condition variables, lock ordering — are exactly the primitives every concurrent program in the world is built on. Operating systems, databases, web servers, game engines: they're all running this same pattern, just at much larger scale.

> *Submission is via the Google Form your mentor will share. See* `week-2/README.md` *for the checklist of files to have ready and what to include in your report.*